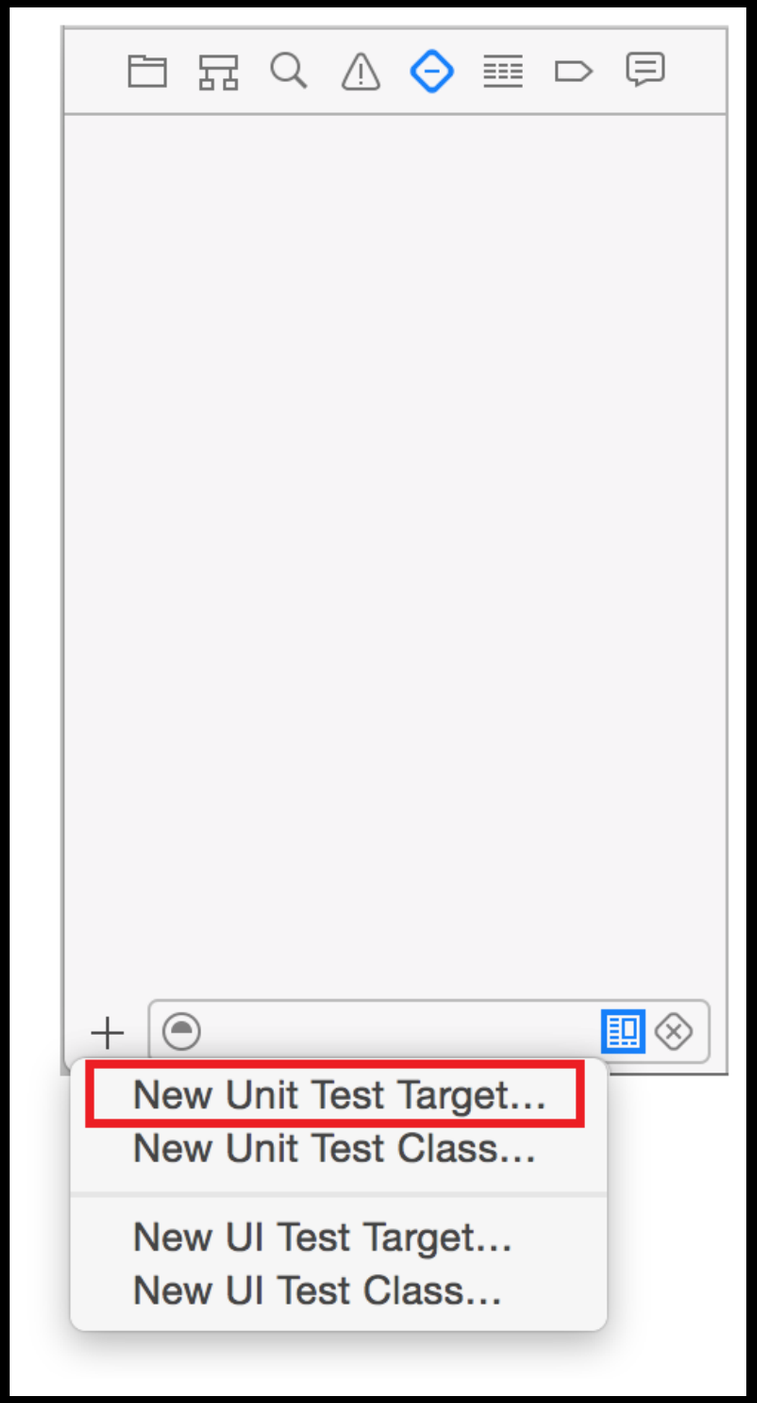
XCTest Sharing

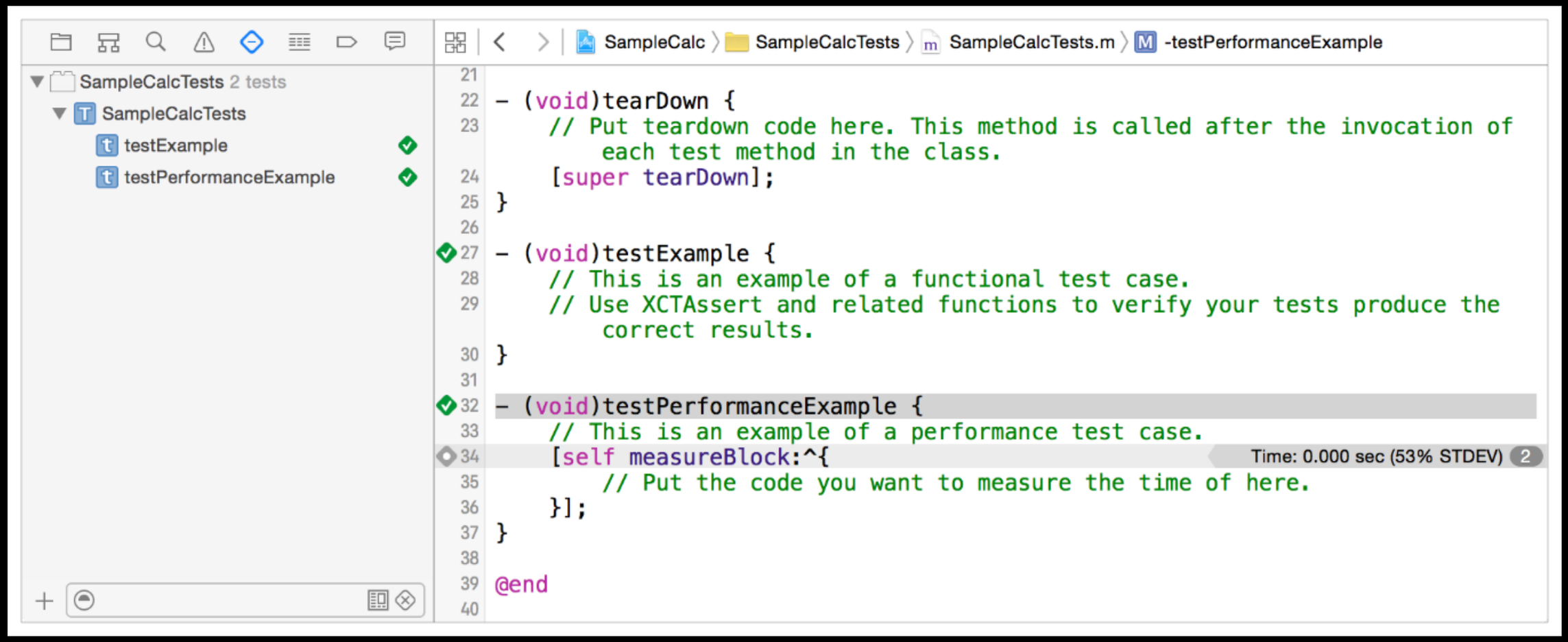
Unit testing is primarily concerned with forming good components that behave as expected and interact with other components as expected.

Add Test Target

- New app, framework, and library projects created in Xcode 5 or later are preconfigured with a test target.

- Test targets can be added manually in Test navigator.





- Using Test navigator to run tests running in one class, one function or the whole test target and see the result.

Setup and teardown: for each test method

```
override func setUp() {
    super.setUp()
    // Put setup code here. This method is called before the invocation of each test method in the class.
}
```

```
override func tearDown() {
    // Put teardown code here. This method is called after the invocation of each test method in the class.
    super.tearDown()

    // When tests completed, remove all stubs.
    OHHTTPStubs.removeAllStubs()
}
```

- Performance testing is useful in scenario like network request. The test result of performance testing is based on the ‘baseline’ we set.

```
64      }
65
66      func testPerformanceExample() {
67          // This is an example of a performance test case.
68          self.measure {
69              // Put the code you want to measure the time of here
70              self.testGetTextAd()
71          }                                          Time: 0.025 sec (98% better)
72      }
73
```

Test asynchronous task

Use XCTestExpectation:

```
func testGetTextAd(){
    let ins = AdManager.defaultInstance

    let expectation: XCTestExpectation = XCTestExpectation()

    var textAd: TextAd?

    ins.getTextAd(language: .ChineseTraditional, success: { (json) in
        textAd = json
        expectation.fulfill()
    }, failure:{
        (error) in
        print(error)
    })
    wait(for: [expectation], timeout: 1000)

    XCTAssertEqual(textAd?.title,
    XCTAssertNotNil(textAd?.url)

}
```

Library usage: OHHTTPStubs (mock HTTP response)

Library usage: Cuckoo (mock general objects)

XCUITest Sharing

UI tests approach testing apps from the external surface, as the users experience it.

UI Recording and Replay

The general pattern of a UI test for correctness is as follows:

- Use an XCUIElementQuery to find an XCUIElement.
- Synthesize an event and send it to the XCUIElement.
- Use an assertion to compare the state of the XCUIElement against an expected reference state.